



서울시립대학교

Lec 3 - Interrupt & ISR (IRQHandler)

전력전자연구실 교수 이 승 환



✓ Interrupt 개념 :

- MCU는 외부 소자와 연결하기 위해서 다양한 주변 장치들을 제공.
- 이들 주변 장치들이 갑자기, Core의 정상적인 code 명령어 해석 작업보다 우선순위가 높은 업무를 발생시킬 수 있는데, 이것을 interrupt라고 한다.

✓ Interrupt Controller :

- 여러 주변 장치들에서 발생한 interrupt들 중에서 어느 것을 먼저 수행할지 우선순위를 결정
- 발생한 interrupt들에 대한 우선순위는 기본적으로 미리 정해져 있다.
- 최종적으로 발생한 interrupt들 중에서 어느 하나만 선정
- 선정된 interrupt만 Core의 정상적인 code 명령어 해석 작업을 잠시 멈추고, 처리됨.
- 이와 같은 작업을 수행하는 Digital 논리 회로를 Interrupt Controller라고 한다

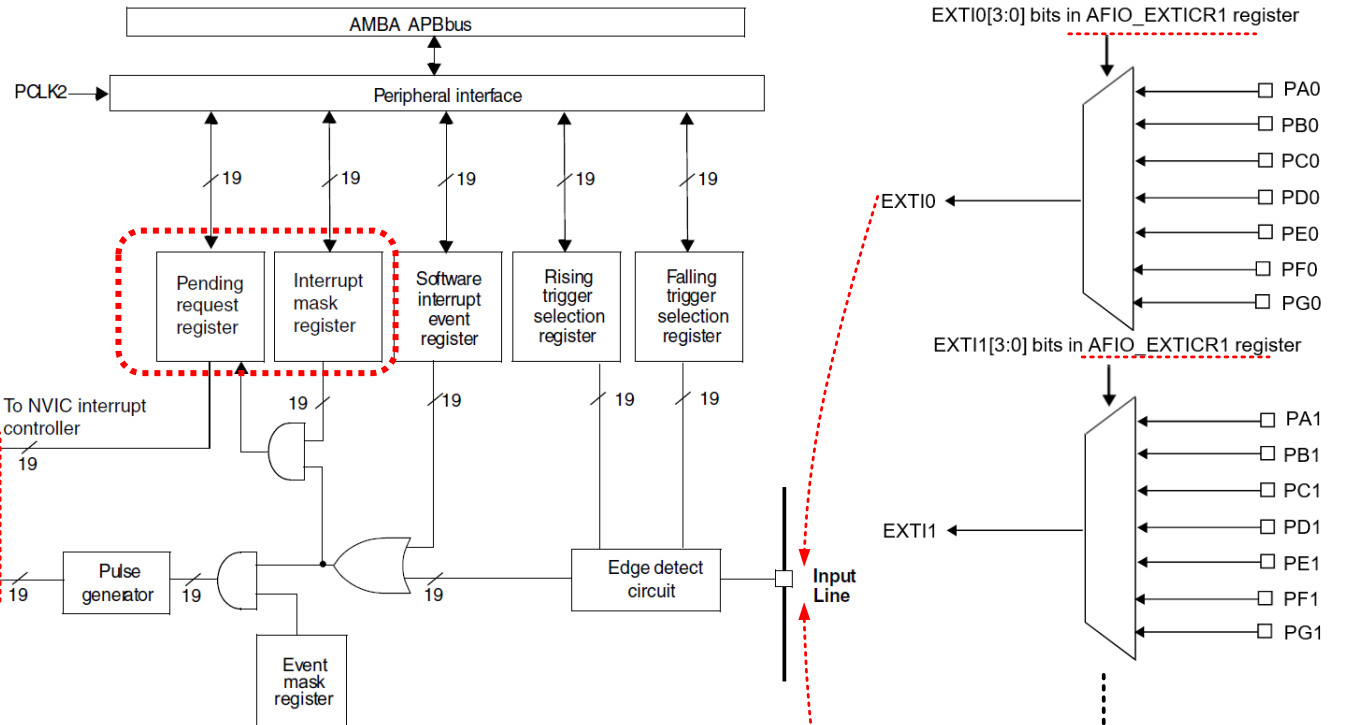
✓ NVIC :

- STM32 MCU처럼 Cortex-M core family를 사용하는 경우에는 ARM Inc.에서 개발한 NVIC(Nested Vectored Interrupt Controller)라는 공통적인 interrupt controller를 모두 사용.

✓ Interrupt 처리 방법 :

- 기본적으로 interrupt가 발생하면, 발생한 interrupt가 호출할 함수 즉, Interrupt Service Routine(ISR)을 미리 coding하여 놓는다.
- 미리 발생할 interrupt 각각에 대해 호출될 함수 즉, ISR의 시작 번지들을 순서대로 저장해 놓은 것을 interrupt vector table이라고 한다.
- 임의의 interrupt가 발생하면, interrupt controller 즉, NVIC가 우선 순위와 round robin 방식에 근거하여 최종적으로 선택한 해당 ISR을 core가 현재의 작업을 중단하고, 수행한다.
- 임의의 interrupt 발생에 따른 ISR에서 수행할 내용이 많다면, 이것도 software 적으로 event를 발생하여 해당 callback 함수를 호출하여 수행한다.

외부 Interrupt 개념과 처리 방법 - 1



(2) STM32 내부에서 NVIC로 보내기 위한 event화 ← (1) GPIO를 통한 외부 interrupt

5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
22	29	settable	CAN1_SCE	CAN1 SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0

(3) NVIC interrupt 우선 순위 리스트

GPIO Mode and Configuration

Configuration

Group By Peripherals

☒ GPIO ☒ Single Mapped Signals ☒ RCC ☒ SYS ☒ USART ☒ NVIC

Search Signals

Search (Ctrl+F)

☐ Show only Modified Pins

Pin Na...	Signal on ...	GPIO out...	GPIO mode	GPIO Pull...	Maximum...	User Label	Modified
PA5	n/a	Low	Output P...	Pull-down	Low	LD2	☒
PC13-TA...	n/a	n/a	External I...	Pull-up	n/a	EXTI_13	☒

PC13-TAMPER-RTC Configuration :

GPIO mode: External Interrupt Mode with Falling edge trigger detection

GPIO Pull-up/Pull-down: Pull-up

User Label: EXTI_13

PC13-TAMPER-RTC Configuration :

GPIO mode: External Interrupt Mode with Falling edge trigger detection

GPIO Pull-up/Pull-down: Pull-up

User Label: EXTI_13

External Interrupt Mode with Falling edge trigger detection

External Interrupt Mode with Rising edge trigger detection

External Interrupt Mode with Falling edge trigger detection

External Event Mode with Rising edge trigger detection

External Event Mode with Falling edge trigger detection

External Event Mode with Rising/Falling edge trigger detection

- ✓ STM32F103 MCU 내부에서 발생한 외부 GPIO interrupt를 처리하는 방법.
 - EXTI0부터 EXTI4는 각각 자신만의 interrupt line을 갖는다.
 - EXTI5부터 EXTI9까지는 EXTI9_5 line 하나에 공통으로 연결되어 있다.
 - EXTI10부터 EXTI15까지도 EXTI15_10 line 하나에 공통으로 연결되어 있다.
 - 결국, EXTI5부터 EXTI9까지 임의의 interrupt가 발생하면, EXTI9_5과 관련된 공통의 ISR이 생성되고 이것을 이용해야 한다는 것을 의미.
 - GPIO 외부 interrupt callback 함수인 HAL_GPIO_EXTI_Callback()에서 발생한 interrupt가 EXTI5 interrupt인지 EXTI9 interrupt를 구분한다.
 - Interrupt 우선순위는 숫자가 낮을수록 높다. 즉, 우선 순위 13번을 갖는 EXTI0 interrupt와 14번을 갖는 EXTI1 interrupt가 동시에 발생하였다면, EXTI0 interrupt가 우선 순위가 높으므로 먼저 수행된다.

- ✓ AFIO_EXTICR1..4 register의 EXTIx[3:0] 값을 이용하여 interrupt source PA/G0~PA/G15 16개 입력을 선택. 그러므로, PA0과 PG0을 동시에 interrupt source로 사용하면 안 됨! - 교재 109 페이지 부터 참조!

9.4.3 External interrupt configuration register 1 (AFIO_EXTICR1)

Address offset: 0x08

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20
Reserved											
15	14	13	12	11	10	9	8	7	6	5	4
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x= 0 to 3)

These bits are written by software to select the source input for EXTI external interrupt.
Refer to [Section 10.2.5: External interrupt/event line mapping](#)

0000: PA[x] pin
0001: PB[x] pin
0010: PC[x] pin
0011: PD[x] pin
0100: PE[x] pin
0101: PF[x] pin
0110: PG[x] pin

9.4.6 External interrupt configuration register 4 (AFIO_EXTICR4)

Address offset: 0x14

Reset value: 0x0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved

Bits 15:0 **EXTIx[3:0]**: EXTI x configuration (x= 12 to 15)

These bits are written by software to select the source input for EXTIx external interrupt.

0000: PA[x] pin
0001: PB[x] pin
0010: PC[x] pin
0011: PD[x] pin
0100: PE[x] pin
0101: PF[x] pin
0110: PG[x] pin

Cortex-M Interrupt Vector Table - 1

Position	Interrupt type	IRQ Num	번 지	설명
	NA	NA	0x0000_0000	Initial Stack Pointer.
우선 순위 조정 불가	Reset		0x0000_0004	Reset vector
	NMI	-14	0x0000_0008	Non-Maskable Interrupt
	Hard Fault	-13	0x0000_000C	만일, 대응하는 exception handler가 enabling되어 있지 않은 상태에서 system exception이 발생한 경우.
	MemManage fault	-12	0x0000_0010	MMFSR 참조
4	Bus fault	-11	0x0000_0014	data/prefetch abort에 해당. BFSR 참조.
6	Usage fault	-10	0x0000_0018	UFSR 참조.
7~10	Reserved	NA		Reserved
11	SVCall	-5	0x0000_002C	System Service Call
12	Debug Monitor		0x0000_0030	breakpoints, watchpoints, etc
13	Reserved	NA		Reserved
14	PendSV	-2	0x0000_0038	System device를 위한 pendable request
15	SYSTICK	-1	0x0000_003C	System Tick timer
16	WWDG	0	0x0000_0040	Window Watchdog interrupt
17	PVD	1	0x0000_0044	PVD through EXTI Line detection interrupt
..	...	...		...
22	EXTI0	6	0x0000_0058	GPIO 외부 interrupt #0

Cortex-M Interrupt Vector Table - 2

- ✓ STM32 MCU interrupt vector table은 유사하다.
 - STM32F103와 STM32L476 interrupt vector table 비교.

0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
1	8	settable	PVD_PVM	PVD/PVM1/PVM2/PVM3/PVM4 through EXTI lines 16/35/36/37/38 interrupts	0x0000 0044
2	9	settable	RTC_TAMP_STAMP /CSS_LSE	RTC Tamper or TimeStamp /CSS on LSE through EXTI line 19 interrupts	0x0000 0048
3	10	settable	RTC_WKUP	RTC Wakeup timer through EXTI line 20 interrupt	0x0000 004C
4	11	settable	FLASH	Flash global interrupt	0x0000 0050
5	12	settable	RCC	RCC global interrupt	0x0000 0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000 0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000 005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000 0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000 0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000 0068

(a) STM32L476 vector table

0	7	settable	WWDG	Window Watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
3	10	settable	RTC	RTC global interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068

(b) STM32F103 vector table

✓ Coding 목 표 :

- PC13번 pin에 연결되어 있는 Blue Switch를 GPIO 외부 interrupt 입력으로 받고, 누를 때마다 PA5번 pin에 연결되어 있는 LD2 LED가 켜지고, 꺼지는 것을 반복하도록 Nucleo-STM32F103RB 보드에 구현.

✓ Coding을 위한 구성 :

- PA5번 port를 GPIO Output port로 설정하여 LD2(Green)를 구동하도록 한다.
- PC13번 port를 GPIO 외부 interrupt Input port로 설정하여 Blue Switch 입력을 받도록 한다.

✓ 학습의 의미 :

- GPIO port를 이용한 외부 interrupt에 대한 개념을 구현하고, 이해한다.
- 인터럽트 관련 흐름도를 학습한다.
- Callback 함수에 대한 개념을 학습한다.

✓ Coding 목 표 :

- PC13번 pin에 연결되어 있는 Blue Switch를 GPIO 외부 interrupt 입력으로 받고, 누를 때마다 PA5번 pin에 연결되어 있는 LD2 LED가 켜지고, 꺼지는 것을 반복하도록 Nucleo-STM32F103RB 보드에 구현.

✓ Coding을 위한 구성 :

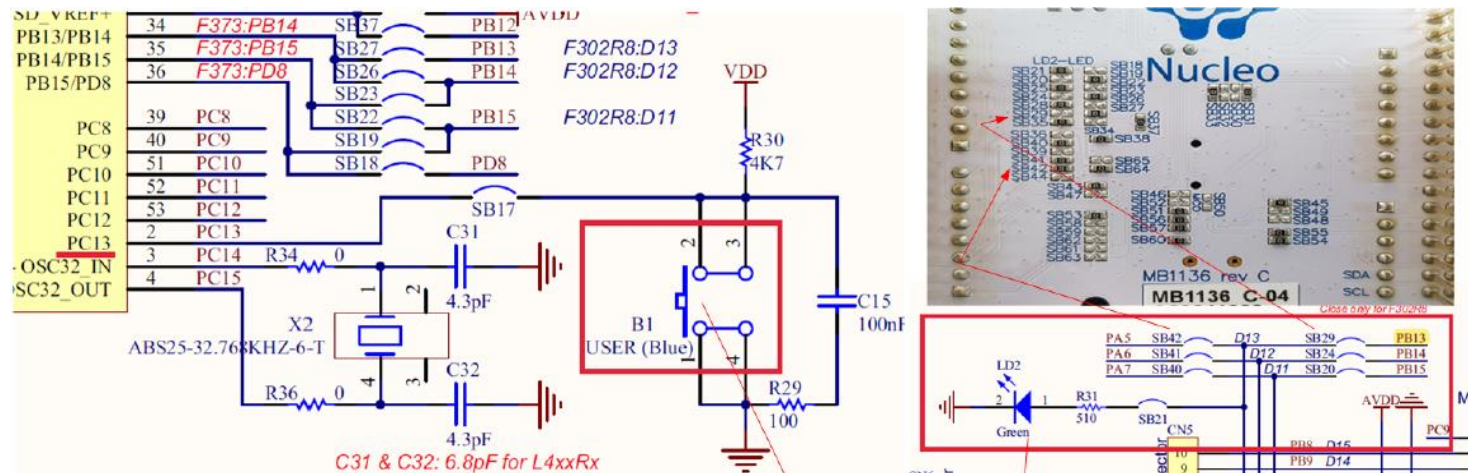
- PA5번 port를 GPIO Output port로 설정하여 LD2(Green)를 구동하도록 한다.
- PC13번 port를 GPIO 외부 interrupt Input port로 설정하여 Blue Switch 입력을 받도록 한다.

✓ 학습의 의미 :

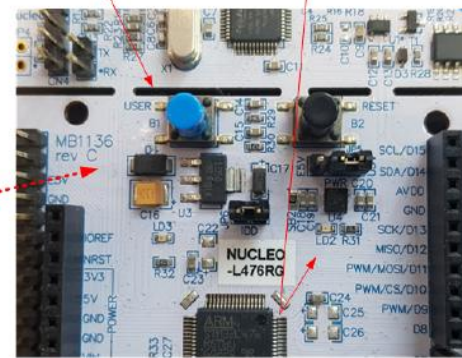
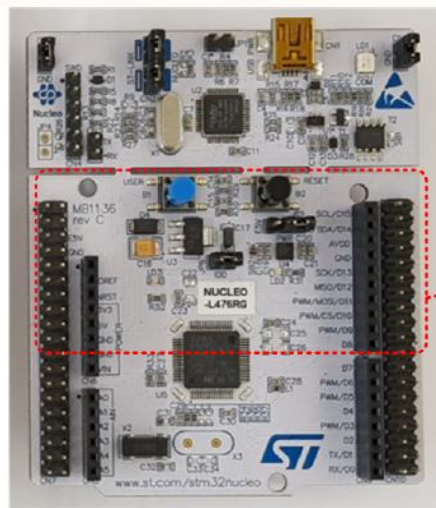
- GPIO port를 이용한 외부 interrupt에 대한 개념을 구현하고, 이해한다.
- 인터럽트 관련 흐름도를 학습한다.
- Callback 함수에 대한 개념을 학습한다.

✓ 64pins STM32F103 Nucleo-64 회로도 :

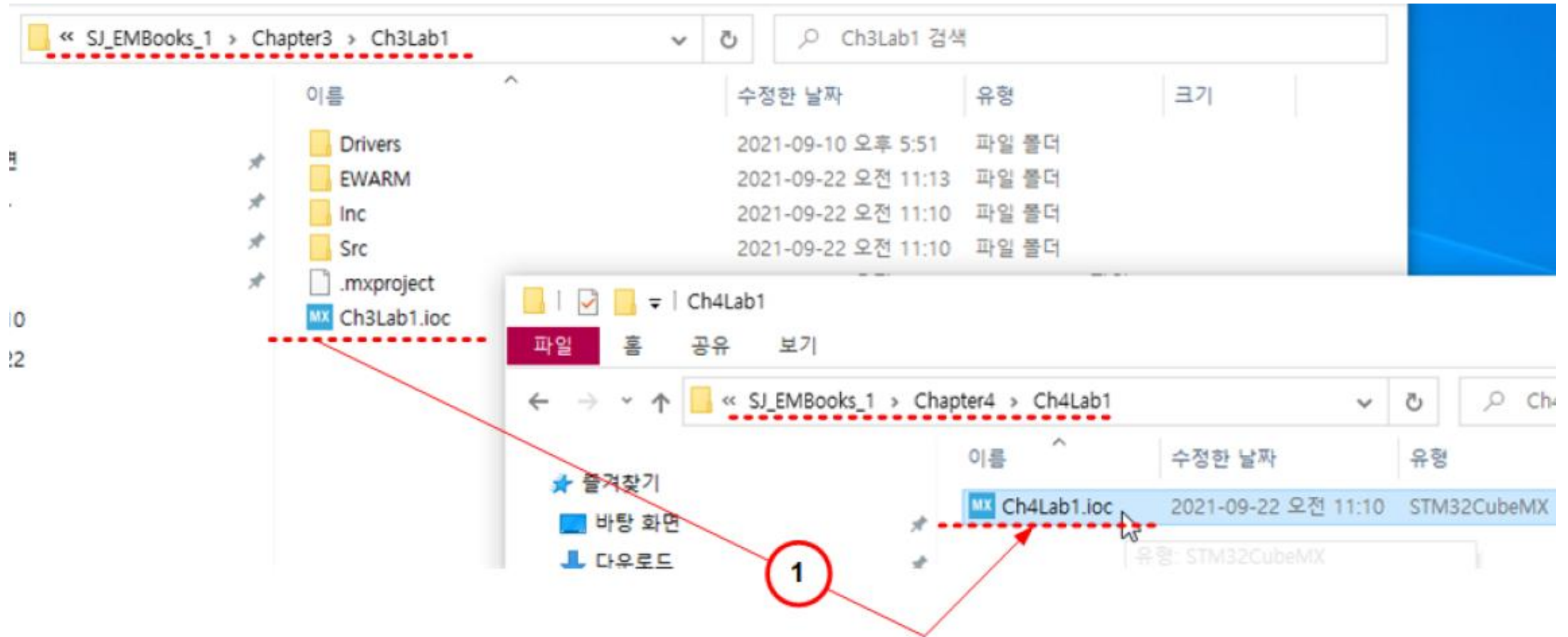
- PC13번 pin은 default High이며 GPIO 외부 interrupt 입력, 누를 때마다 PA5번 pin에 연결되어 있는 LD2 LED가 켜지고, 꺼지는 것을 반복.



Switch	LED2
1	0(LED Off)
0(Push)	1(LED On)



- ✓ Ch3Lab1 folder에 있는 Ch3Lab1.ioc의 파일 이름을 ①번과 같이 Ch4Lab1.ioc 파일로 이름 변경.



✓ PC13번 pin을 외부 interrupt로 설정하는 방법.

- PC13번 pin default High Falling edge trigger
- Default 입력은 회로도와 동일하게 Pull-up으로 설정!

Home > STM32L476RGTx - NUCLEO-L476RG > Ch4Lab1.ioc - Pinout & Configuration > GENERATE CODE

Pinout & Configuration | Clock Configuration | Project Manager | System view

Software Packs | Pinout

GPIO Mode and Configuration

Configuration

Group By Peripherals

GPIO Single Mapped Signals RCC SYS USART

Search Signals

Search (Ctrl+F)

Show only Modified Pins

Pin	Signal	GPIO	GPIO	GPIO	Maxim	Fast M	User L	Modified
PA5	n/a	Low	Output...	Pull-do...	Low	n/a	LD2	✓
PC13	n/a	n/a	Input ...	No pull...	n/a	n/a	Switch	✓

PC13 Configuration :

GPIO mode: Input mode

GPIO Pull-up/Pull-down: No pull-up and no pull-down

User Label: Switch

PC13 Configuration :

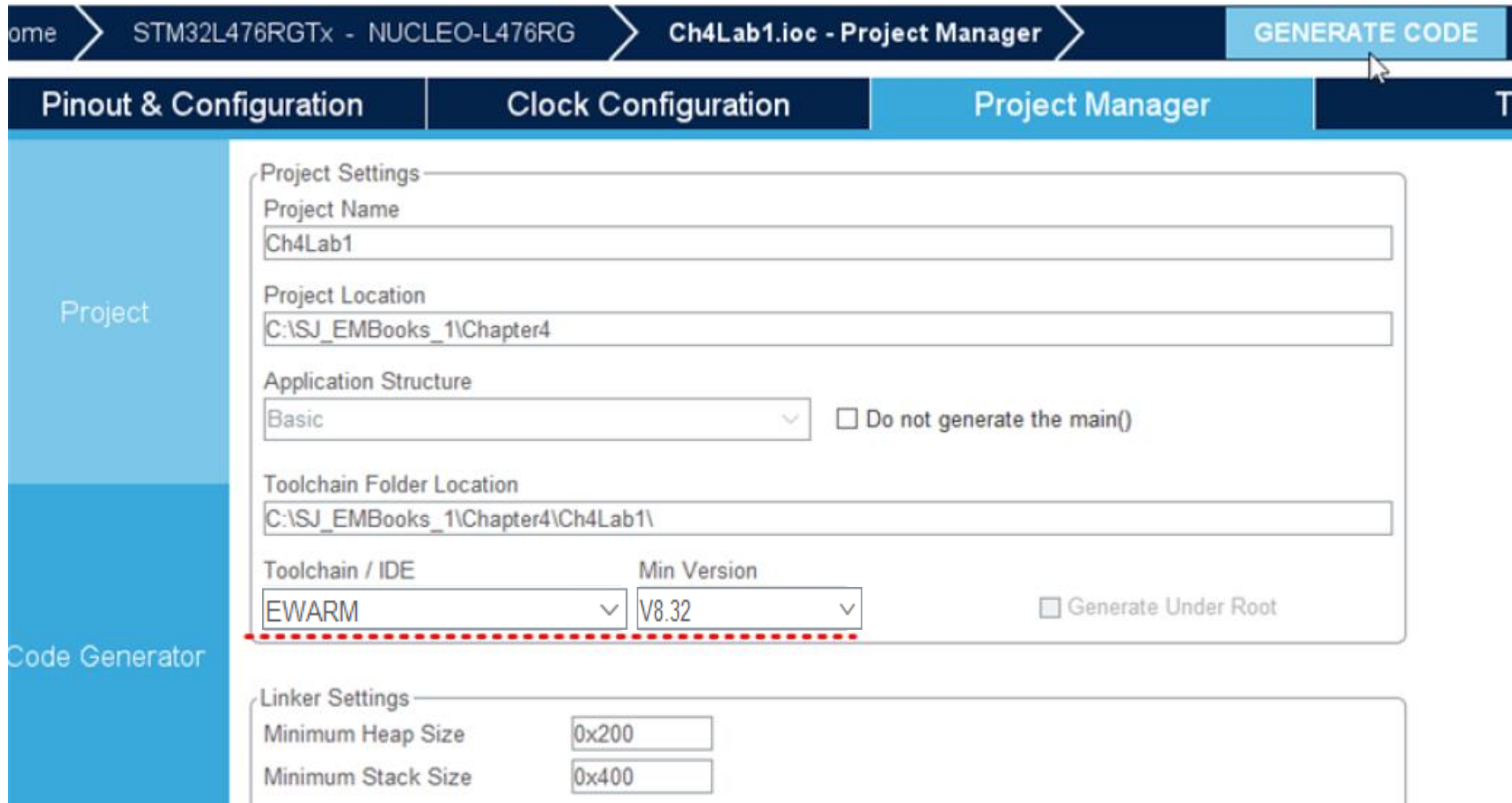
GPIO mode: External Interrupt Mode with Falling edge trigger detection

GPIO Pull-up/Pull-down: Pull-up

User Label: Pull-up

GPIO_EXTI13

- ✓ Toolchain으로 EWARM 선택하고, GENERATE CODE button click!



The screenshot shows the STM32CubeIDE Project Manager interface. The top navigation bar includes 'ome', 'STM32L476RGTx - NUCLEO-L476RG', 'Ch4Lab1.ioc - Project Manager', and a highlighted 'GENERATE CODE' button. Below this, the 'Project Manager' tab is selected. The left sidebar has 'Project' and 'Code Generator' sections. The main area displays 'Project Settings' with the following fields:

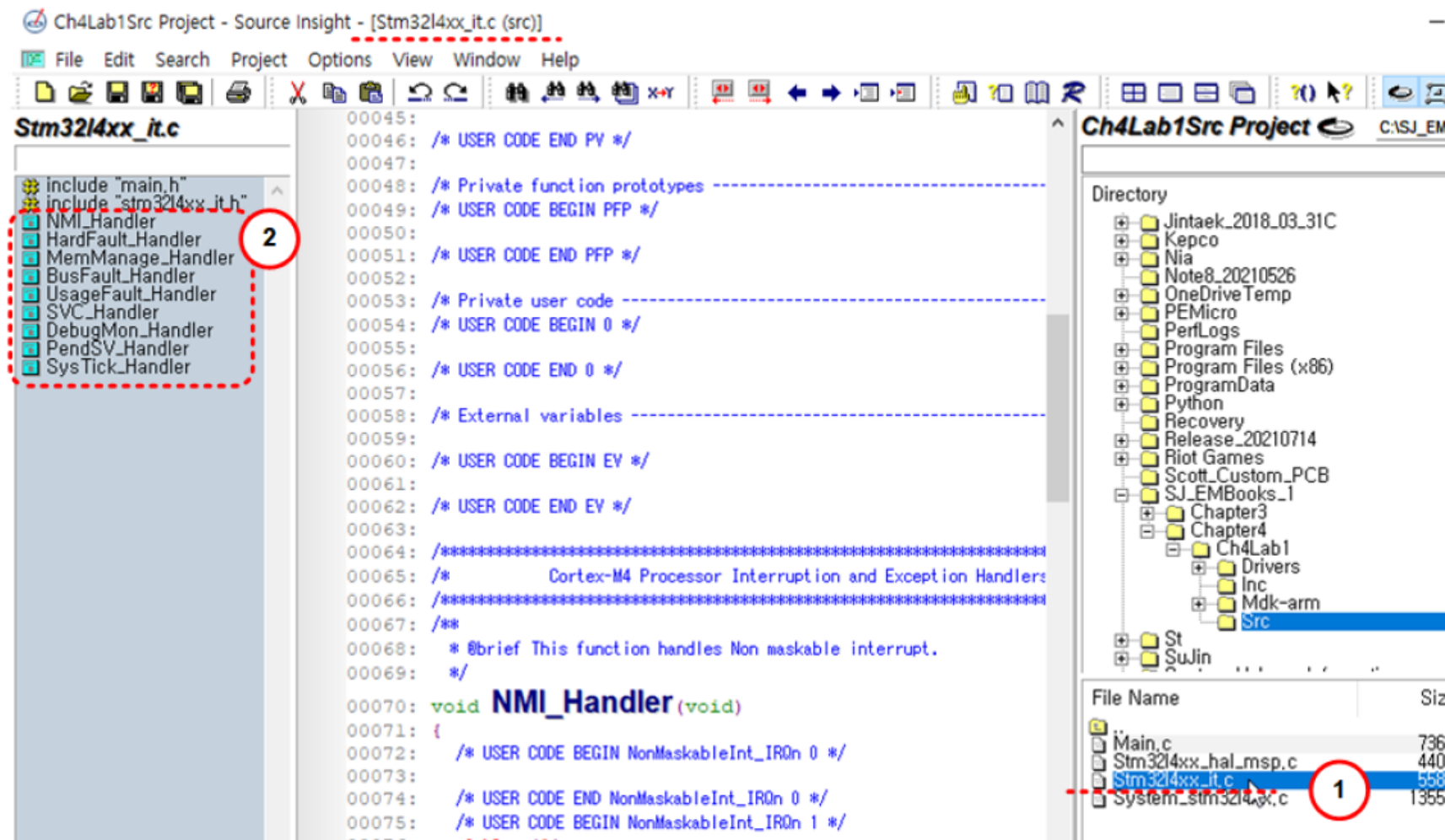
- Project Name:** Ch4Lab1
- Project Location:** C:\SJ_EMBooks_1\Chapter4
- Application Structure:** Basic (dropdown menu) with a checkbox for 'Do not generate the main()'.
- Toolchain Folder Location:** C:\SJ_EMBooks_1\Chapter4\Ch4Lab1\
- Toolchain / IDE:** EWARM (dropdown menu)
- Min Version:** V8.32 (dropdown menu) with a checkbox for 'Generate Under Root'.

Below the 'Project Settings' section, the 'Linker Settings' are visible:

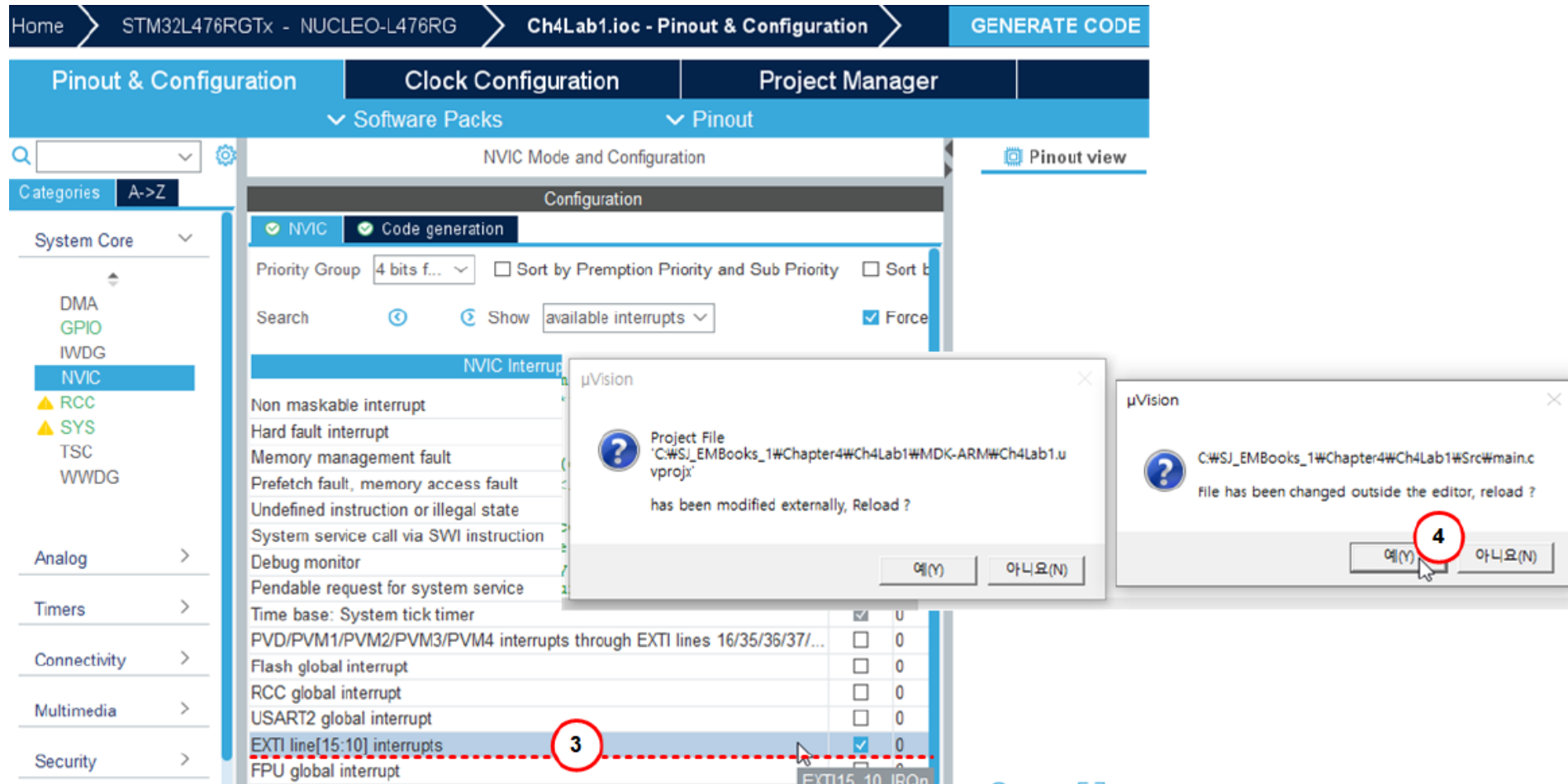
- Minimum Heap Size:** 0x200
- Minimum Stack Size:** 0x400

✓ stm32l4xx_it.c file의 구조 :

- ②번을 통해서 예약된 system interrupt들 확인.
추가한 PC13번 pin의 외부 interrupt에 대한 ISR이 보이지 않는다.
- PC13번 pin은 13 즉, 10과 15 사이이므로 EXTI15_10 interrupt line 함께 사용.



- ③번과 같이 EXTI line[15:10] interrupts 옆의 Enabled checkbox를 선택!
- 그리고 나서, 다시 GENERATE CODE button을 click.



- EXTI15_10_IRQHandler() 함수가 새롭게 추가!

Ch4Lab1Src Project - Source Insight - [Stm32l4xx_it.c (src)]

File Edit Search Project Options View Window Help

Stm32l4xx_it.c

- include "main.h"
- include "stm32l4xx_it.h"
- NMI_Handler
- HardFault_Handler
- MemManage_Handler
- BusFault_Handler
- UsageFault_Handler
- SVC_Handler
- DebugMon_Handler
- PendSV_Handler
- SysTick_Handler
- EXTI15_10_IRQHandler**

```

00187:
00188:  /* USER CODE END SysTick_IRQn 0 */
00189:  HAL_IncTick();
00190:  /* USER CODE BEGIN SysTick_IRQn 1 */
00191:
00192:  /* USER CODE END SysTick_IRQn 1 */
00193: }
00194:
00195: /******
00196:  /* STM32L4xx Peripheral Interrupt Handlers
00197:  /* Add here the Interrupt Handlers for the used peripherals.
00198:  /* For the available peripheral interrupt handler names,
00199:  /* please refer to the startup file (startup_stm32l4xx.s).
00200:  /******
00201:
00202:  /**
00203:   * @brief This function handles EXTI line[15:10] interrupts.
00204:   */
00205:  void EXTI15_10_IRQHandler(void)
00206:  {
00207:    /* USER CODE BEGIN EXTI15_10_IRQn 0 */
00208:
00209:    /* USER CODE END EXTI15_10_IRQn 0 */
00210:    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_13);
00211:    /* USER CODE BEGIN EXTI15_10_IRQn 1 */
00212:
00213:    /* USER CODE END EXTI15_10_IRQn 1 */
00214:  }
00215:
  
```

HAL_GPIO_EXTI_IRQHandler Function in Stm32l4xx_hal_gpio.c (drivers\...\src) at line 513 (9 lines)

```

/*
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x0000u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
  
```

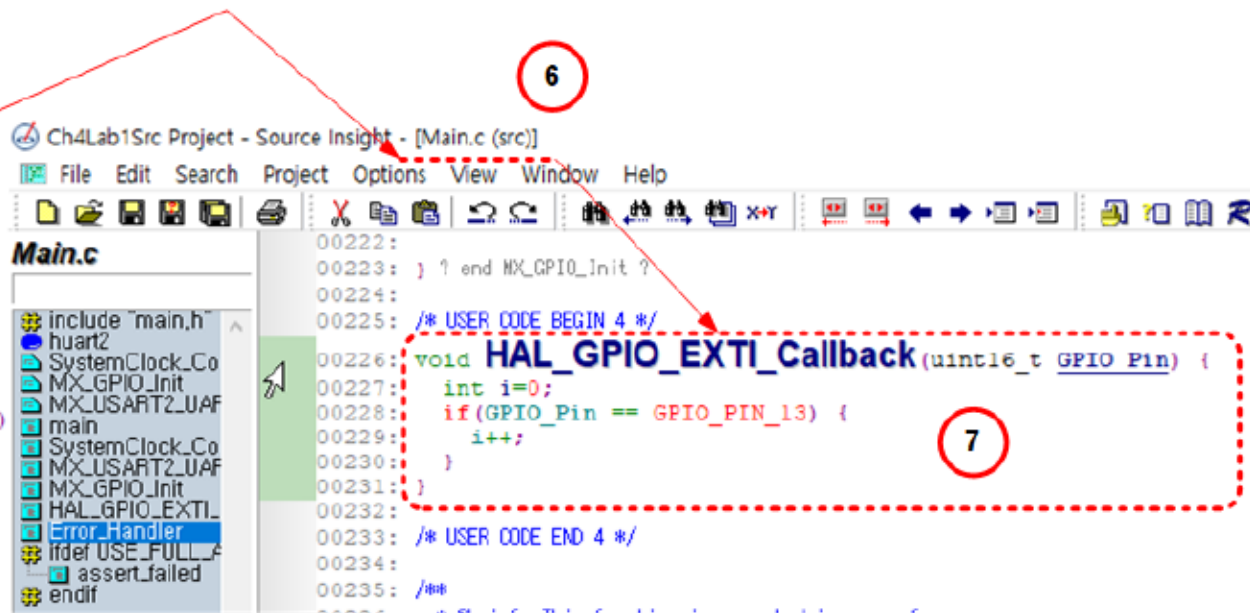
4

- ④번과 같이 HAL_GPIO_EXTI_Callback() 함수의 선언을 찾아보면, 그 앞에 ⑤번과 같이 __weak 라는 keyword가 함수의 prototype 앞에 추가되어 있다.
- 어떠한 함수의 선언 앞에 예약어 __weak를 추가하면, 동일한 함수가 다른 파일에 정의 되어 있지 않다면, 예약어 __weak가 붙은 함수를 수행하고, 만일, 동일한 함수가 다른 파일에 존재한다면, 그 함수를 수행하겠다는 의미.

```

00508: /*
00509:  * @brief Handle EXTI interrupt request.
00510:  * @param GPIO_Pin Specifies the port pin connected to corresponding EXTI line.
00511:  * @retval None
00512:  */
00513: void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
00514: {
00515:     /* EXTI line interrupt detected */
00516:     if ( __HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u )
00517:     {
00518:         HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
00519:         HAL_GPIO_EXTI_Callback(GPIO_Pin);
00520:     }
00521: }
00522:
00523: /*
00524:  * @brief EXTI line detection callback.
00525:  * @param GPIO_Pin Specifies the port pin connected to corresponding EXTI line.
00526:  * @retval None
00527:  */
00528: ⑤ weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
00529: {
00530:     /* Prevent unused argument(s) compilation warning */
00531:     UNUSED(GPIO_Pin);
00532:
00533:     /* NOTE: This function should not be modified, when the callback is needed,
00534:      the HAL_GPIO_EXTI_Callback could be implemented in the user file
00535:  */

```



✓ 실습 방법 :

- main.c 에 다음과 같은 bFlgPC13 int 변수와 HAL_GPIO_EXTI_Callback() 함수 추가

```
/* USER CODE BEGIN PV */
int bFlgPC13;
/* USER CODE END PV */
```

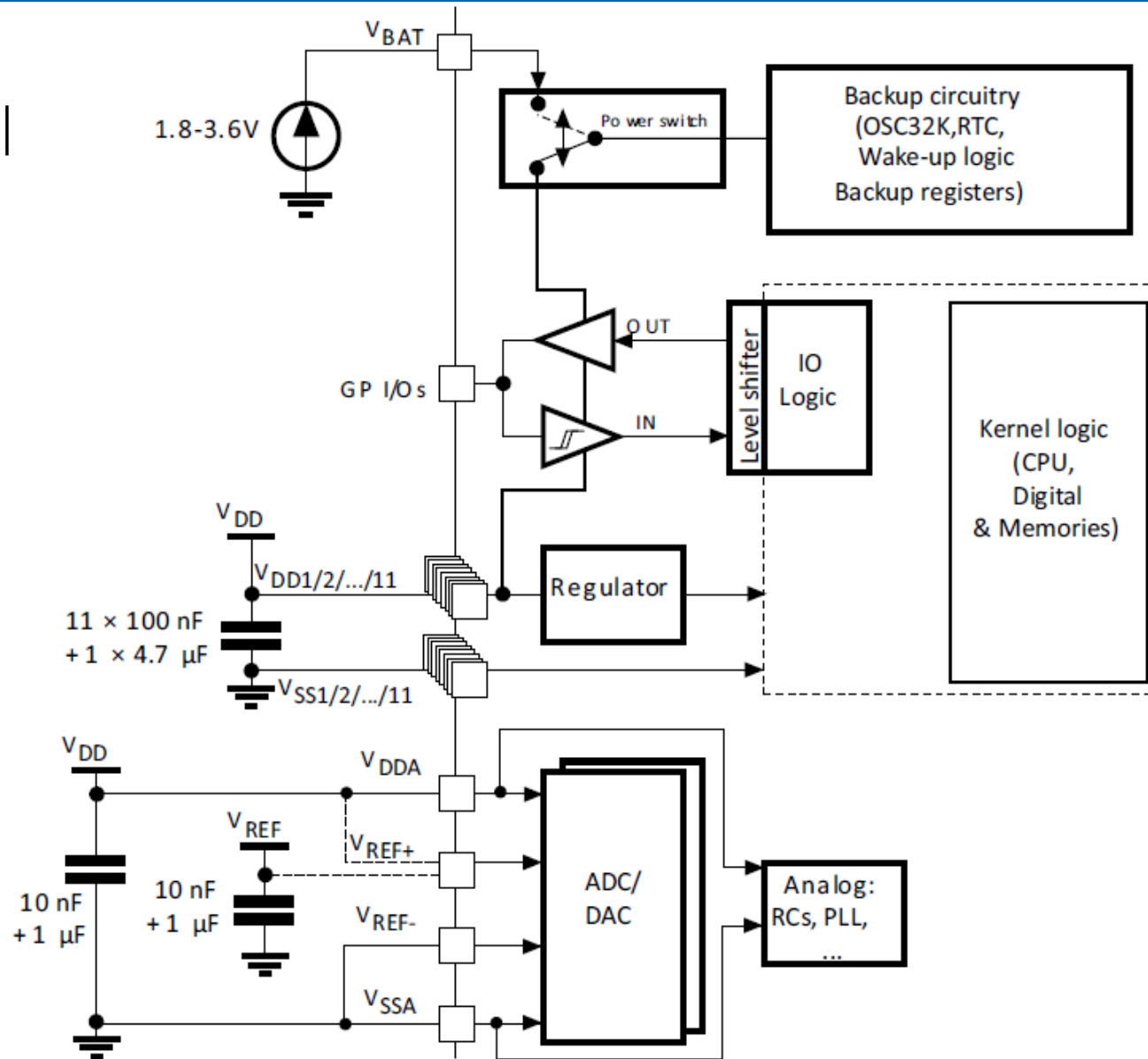
```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    if(GPIO_Pin == GPIO_PIN_13){
        bFlgPC13++;
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
/* USER CODE END 4 */
```

- 실행을 시키고, Nucleo-F103RB 보드에 있는 파란색 switch를 click.
- 전역변수 bFlgPC13의 값이 바뀌는 것과 그에 따른 PA5에 연결되어 있는 LD2 LED가 점멸하는 것을 확인!
- Watch window와 그 밖에 IAR toolchain에서 제공하는 debugging 기능 학습.

✓ 과 제 :

- 실험실내의 Function generator 이용해 0-3V, 2Hz pulse waveform 만들고, 이를 EXTI0 인터럽트 소스로 사용해 Green LED를 2Hz로 On-off 하는 Interrupt Service Routine 만들기 (조교확인필수)

- ✓ 외부에서 공급되는 전원이 정해진 전압 범위인지 감시 :
 - 공급 전원 VDD/VDDA가 2[V] 보다 커지거나 또는 작아지면, POR(Power On Reset) 또는 PDR(Power Down Reset)이 발생하여 자동으로 NRST pin을 low로 구동한다.
 - 내부 Digital logic을 위한 전원 VDD = 2.0[V]~3.6[V],
 - 내부 Analog logic을 위한 전원 VDDA=2.0[V]~3.6[V] 단, ADC 또는 DAC를 사용하는 경우에는 최소 VDDA=2.4[V]
 - VDDA는 VDD에 연결하여 사용해야 한다. 그러므로, ADC 또는 DAC를 사용하는 경우에는 2.4[V]가 최소 전원.
 - VBAT =1.8[V]~3.6[V], VDD가 없을 때에 backup register와 RTC를 위한 전원.
 - STM32F103RC datasheet의 17페이지 참조.



✓ 전원 감시 방법 :

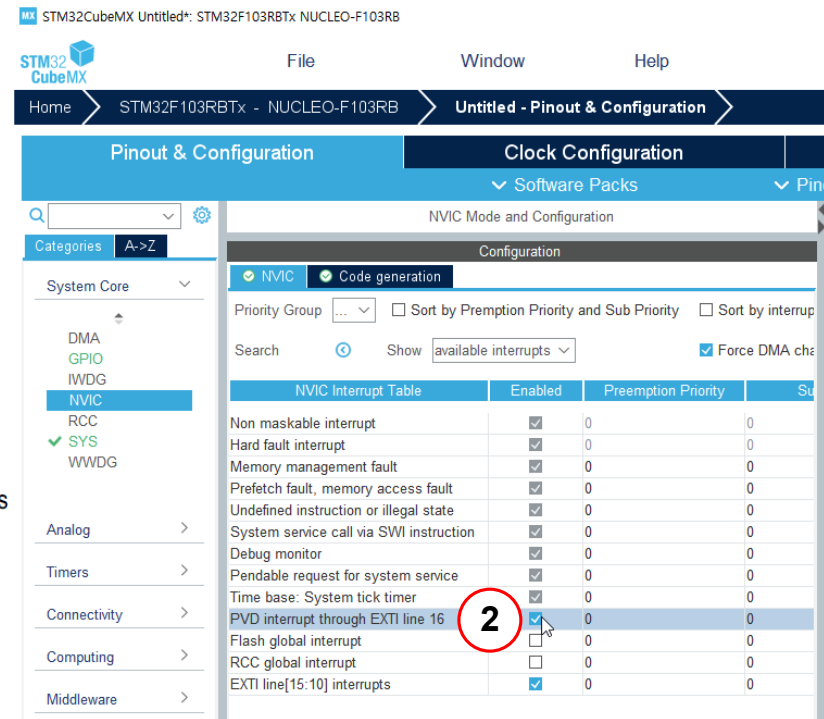
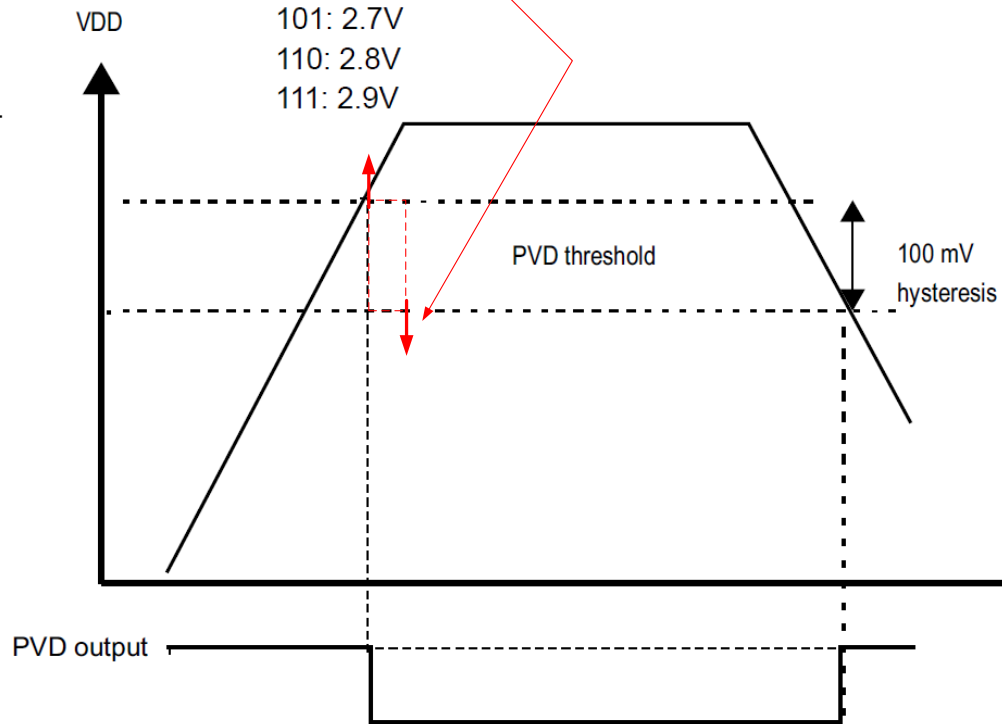
- PVD(Programmable Voltage Detector)는 VDD/VDDA 전원 공급을 PLS[2:0] bit field에 지정한 threshold와 비교하여 EXTI16을 발생시킨다.

Bits 7:5 **PLS[2:0]**: PVD level selection.

These bits are written by software to select the voltage threshold detected by the Power Voltage Detector

000: 2.2V
001: 2.3V
010: 2.4V
011: 2.5V
100: 2.6V
101: 2.7V
110: 2.8V
111: 2.9V

1



5.4.1

Power control register (PWR_CR)

Address offset: 0x00

Reset value: 0x0000 0000 (reset by wakeup from Standby mode)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17
Reserved														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Reserved								DBP	PLS[2:0]		PVDE	CSBF	CWUF	PDDS
								rw	rw	rw	rw	rw	rc_w1	rc_w1

Bit 4 **PVDE**: Power voltage detector enable.

This bit is set and cleared by software.

0: PVD disabled

1: PVD enabled

5.4.2

Power control/status register (PWR_CSR)

Address offset: 0x04

Reset value: 0x0000 0000 (not reset by wakeup from Standby mode)

Additional APB cycles are needed to read this register versus a standard APB read.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17
Reserved														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1
Reserved								EWUP	Reserved				PVDO	SBF
								rw					r	r

Bit 2 **PVDO**: PVD output

This bit is set and cleared by hardware. It is valid only if PVD is enabled by the PVDE bit.

0: V_{DD}/V_{DDA} is higher than the PVD threshold selected with the PLS[2:0] bits.

1: V_{DD}/V_{DDA} is lower than the PVD threshold selected with the PLS[2:0] bits.

```

void HAL_Msplnit(void)
{
    /* USER CODE BEGIN Msplnit 0 */
    PWR_PVDTypeDef sConfigPVD;

    /* USER CODE END Msplnit 0 */

    __HAL_RCC_AFIO_CLK_ENABLE();
    __HAL_RCC_PWR_CLK_ENABLE();

    /* System interrupt init*/

    /* Peripheral interrupt init */
    /* PVD_IRQn interrupt configuration */
    HAL_NVIC_SetPriority(PVD_IRQn, 0, 0);
    HAL_NVIC_EnableIRQ(PVD_IRQn);

    /** NOJTAG: JTAG-DP Disabled and SW-DP Enabled
    */
    __HAL_AFIO_REMAP_SWJ_NOJTAG();

    /* USER CODE BEGIN Msplnit 1 */

    /* Configure the PVD Level to */
    sConfigPVD.PVDLevel=PWR_PVDLEVEL_7; // PWR_CR_PLS_2V9
    sConfigPVD.Mode= PWR_MODE_IT_RISING_FALLING;
    HAL_PWR_PVDCfg(&sConfigPVD);

    /* Enable the PVD Output */
    HAL_PWR_EnablePVD();

    /* USER CODE END Msplnit 1 */
}
    
```

다양한 전원 공급 감시에 따른
Routine 추가!
예 : 전원공급 이상 기록 등

Stm32f1xx_it.c

```

00197: /* Add here the Interrupt Handlers for the
00198: /* For the available peripheral interrupt h
00199: /* please refer to the startup file (startup.
00200: /******
00201:
00202: /**
00203:  * @brief This function handles PVD inter
00204:
00205: void PVD_IRQHandler(void)
00206: {
00207:     /* USER CODE BEGIN PVD_IRQn 0 */
00208:
00209:     /* USER CODE END PVD_IRQn 0 */
00210:     HAL_PWR_PVD_IRQHandler();
00211:     /* USER CODE BEGIN PVD_IRQn 1 */
00212:
00213:     /* USER CODE END PVD_IRQn 1 */
00214: }

```

HAL_PWR_PVD_IRQHandler Function in Stm32f1xx_hal_pwr.c (drivers\...\src)

```

00580: void HAL_PWR_PVD_IRQHandler(void)
00581: {
00582:     /* Check PWR exti flag */
00583:     if(__HAL_PWR_PVD_EXTI_GET_FLAG() != RESET)
00584:     {
00585:         /* PWR PVD interrupt user callback */
00586:         HAL_PWR_PVDCallback();
00587:
00588:         /* Clear PWR Exti pending bit */
00589:         __HAL_PWR_PVD_EXTI_CLEAR_FLAG();
00590:     }
00591: }

```

Main.c

```

#include "main.h"
#include <stdbool.h>
ADDR_FLASH_PAGE_56
huart2
bFlgPC13
FlashDemoDat
FlashRdDat
SystemClock_Config
MX_GPIO_Init
MX_USART2_UART_Init
read_flash
write_flash
main
SystemClock_Config
MX_USART2_UART_Init
MX_GPIO_Init
HAL_GPIO_EXTI_Callback
HAL_PWR_PVDCallback
read_flash
write_flash
Error_Handler
#ifdef USE_FULL_ASSERT
assert_failed
#endif

```

```

00223:
00224: /* USER CODE BEGIN 4 */
00225: void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
00226:     if(GPIO_Pin == GPIO_PIN_13) {
00227:         bFlgPC13=!bFlgPC13;
00228:     }
00229: }
00230:
00231: void HAL_PWR_PVDCallback(void) {
00232:     write_flash(ADDR_FLASH_PAGE_56,FlashDemoDat,1);
00233: }
00234:
00235: uint32_t read_flash(uint32_t addr)
00236: {
00237:     uint32_t dd = 0;
00238:
00239:     dd = *(__IO uint32_t*) (addr);
00240:
00241:     return dd;
00242: }
00243:
00244: uint8_t write_flash(uint32_t page,uint32_t data,uint8_t cnt)
00245: {
00246:     static FLASH_EraseInitTypeDef EraseInitStruct;
00247:     uint32_t PAGEError;
00248:
00249:     uint8_t i,res;
00250:
00251:     EraseInitStruct.TypeErase = 0x00;
00252:     EraseInitStruct.PageAddress = page;
00253:     EraseInitStruct.NbPages = 1; // 128byte
00254:     HAL_FLASH_Unlock();
00255:     if (HAL_FLASHEx_Erase(&EraseInitStruct, &PAGEError) != HAL_OK) {
00256:         res = 0;
00257:     }
00258:     else{
00259:         for(i=0;i<cnt;i++)
00260:             HAL_FLASH_Program(FLASH_TYPEPROGRAM_WORD,page+(i*4),data);
00261:         res = 1;
00262:     }
00263:
00264:     HAL_FLASH_Lock();
00265:
00266:     return res;
00267: } // end write_flash ?

```

8

9

6

7

✓ 전원 감시와 관련된 실험 방법 :

STM32 ST-LINK Utility

File Edit View Target ST-LINK External Loader Help

Memory display

Address: 0x0800E000 Size: 0x1D2E Data Width: 32 bits

Device Memory @ 0x0800E000 : Binary File

Target memory, Address range: [0x0800E000 0x0800FD2E]

Address	0	4	8	C
0x0800E000	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E010	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E020	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E030	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

```

00096: /* USER CODE BEGIN 2 */
00097: FlashRdDat = read_flash(ADDR_FLASH_PAGE_56);
00098:
00099: /* USER CODE END 2 */
00100:
00101: /* Infinite loop */
00102: /* USER CODE BEGIN WHILE */
00103: while (1)
00104: {
00105:     if(bFlgPC13) {
00106:         HAL_GPIO_WritePin(GPIOA,GPIO_PIN_5,GPIO_PIN_RESET); // LED will be Off
00107:     } else {
00108:         HAL_GPIO_WritePin(GPIOA,GPIO_PIN_5,GPIO_PIN_SET); // LED will be On
00109:     }

```

STM32 ST-LINK Utility

File Edit View Target ST-LINK External Loader Help

Memory display

Address: 0x0800E000 Size: 0x1D2E Data Width: 32 bits

Device Memory @ 0x0800E000 : Binary File

Target memory, Address range: [0x0800E000 0x0800FD2E]

Address	0	4	8	C
0x0800E000	12345678	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E010	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E020	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
0x0800E030	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF

```

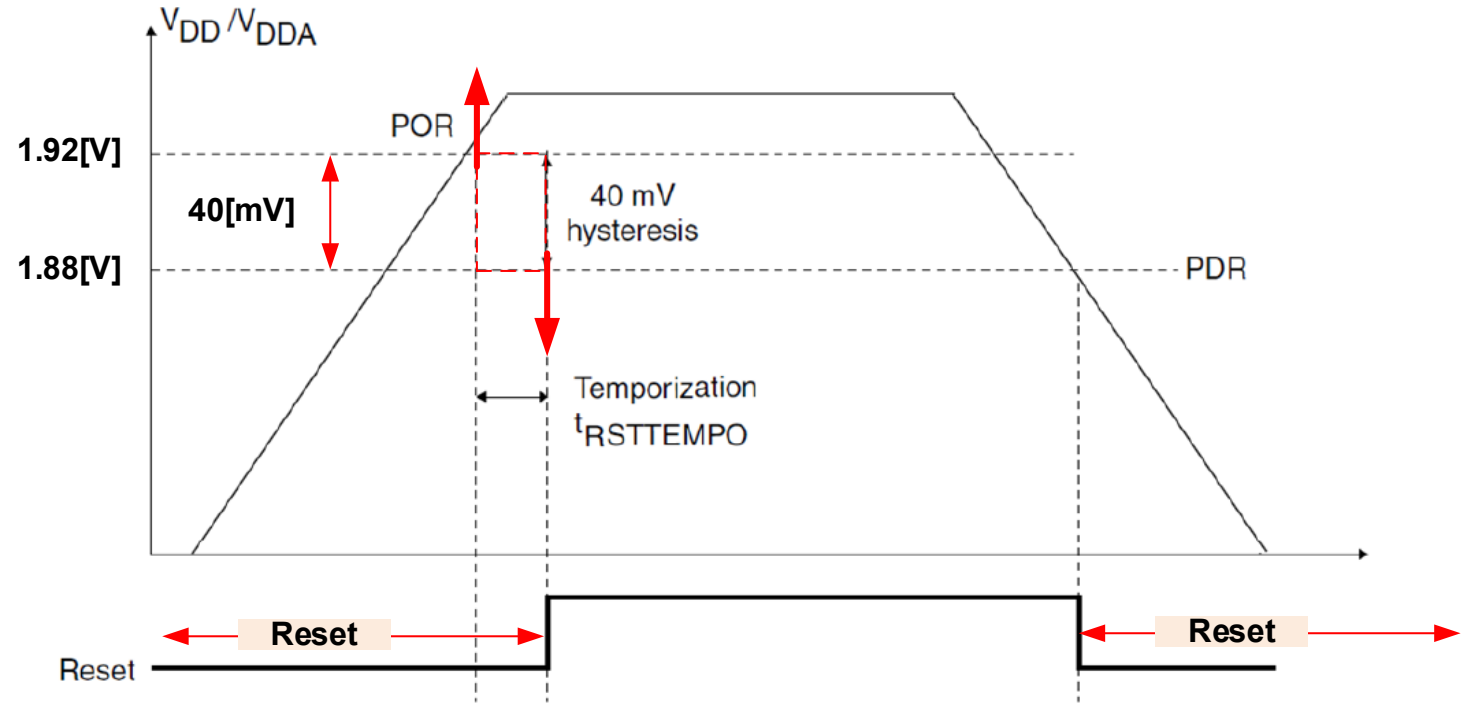
00225: /* USER CODE BEGIN 4 */
00226: void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
00227:     if(GPIO_Pin == GPIO_PIN_13) {
00228:         bFlgPC13=!bFlgPC13;
00229:     }
00230: }
00231:
00232: void HAL_PWR_PVDCallback(void) {
00233:     wrtie_flash(ADDR_FLASH_PAGE_56,FlashDemoDat,1);
00234: }
00235:

```

12

✓ POR/PDR :

- VPDRhyst : PDR hysteresis 는 40[mV] Power On/Off에 대한 reset을 전압의 값이 아닌 범위로 설정하는 것을 Hysteresis라고 한다.
- DC Power Supply를 이용한 실험을 수행 (조교확인필수)



Symbol	Parameter	Conditions	Min.	Typ.	Max.	Unit
$V_{POR/PDR}^{(1)}$	Power on/power down reset threshold	Falling edge	1.8 ⁽²⁾	1.88	1.96	V
		Rising edge	1.84	1.92	2.0	V
$V_{PDRhyst}^{(1)}$	PDR hysteresis	-	-	40	-	mV
$t_{RSTTEMPO}^{(3)}$	POR reset temporization	-	1.5	2.5	4.5	ms

✓ 전력 mode :

- 일반 F family MCU인 STM32F10x과 저전력 L family MCU인 STM32L496 family
- F family는 다음과 같은 전력 mode로 동작시킬 수 있다.
 - Run Mode :
 - reset 이후 MCU는 기본적으로 run mode로 동작한다.
 - Low Power Mode :
 - 일반적으로 다음과 같은 3 종류의 저전력 mode를 사용할 수 있다.
- ① Sleep Mode :
 - CPU clock 정지, 모든 주변 장치들은 정상 동작. interrupt 또는 event가 발생하면 CPU를 깨울 수 있음.
- ② Stop Mode :
 - 모든 clock들은 정지된다. SRAM과 registers의 데이터 값들은 보존 된다.
 - EXTI에서 interrupt가 발생하면 CPU를 깨울 수 있다.
- ③ Standby Mode :
 - 모든 clock들은 정지된다. backup 영역, backup SRAM과 대기 모드 회로내의 registers를 제외한 SRAM과 registers의 데이터 값들은 모두 지워진다. 다음의 경우에 standby mode에서 빠져나올 수 있다. 즉, 외부 reset(NRST), IWDG reset, RTC alarm/wakeup/tamper/time stamp events, WKUP pin에서 rising edge 발생.

✓ 전력 mode :

- L family는 다음과 같은 전력 mode로 동작시킬 수 있다.
 - Run Mode :
 - reset 이후 MCU는 기본적으로 run mode로 동작한다.
 - Low Power Mode :
 - 일반적으로 다음과 같은 7 종류의 저전력 mode를 사용할 수 있다.
 - ㉠ Sleep Mode :
 - ㉡ Low power run Mode :
 - ㉢ Low power sleep Mode :
 - ㉣ Stop 0, Stop 1, 그리고, Stop 2 Mode :
 - ㉤ Standby Mode :
 - ㉥ Shutdown Mode :

Features

- Ultra-low-power with FlexPowerControl
 - 1.71 V to 3.6 V power supply

[그림 5.2-1] STM32L family.